MISA
TIN CẬY - TIỆN ÍCH - TẬN TÌNH

30 Năm PHỤNG SỰ XÃ HỘI
1994 - 2024

ĐẢM BẢO HIỆU NĂNG TẦNG DATABASE

DBO Team



MỤC LỤC

1 DATABASE INDEX

2 EXECUTION PLAN

3 THIẾT KẾ DATABASE ĐẢM BẢO HIỆU NĂNG

4 THI CÔNG DATABASE ĐẢM BẢO HIỆU NĂNG

5 MONITORING

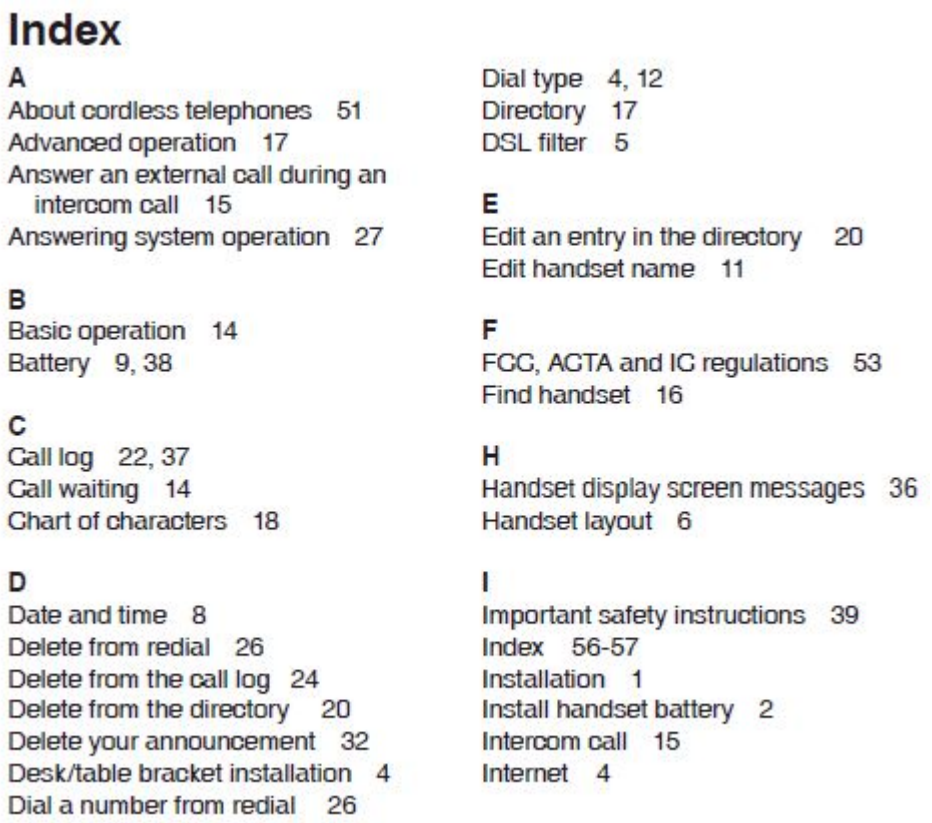
PHẦN 1

DATABASE INDEX

Giới thiệu

- Database index là một cấu trúc dữ liệu giúp cải thiện tốc độ truy vấn dữ liệu và giảm lượng yêu cầu đọc dữ liệu từ ổ đĩa

Có thể hình dung index giống như mục lục một cuốn sách. Khi cần tìm nội dung trong sách, người đọc sẽ tìm ở mục lục trước để biết nội dung cần tìm ở trang nào. Sau đó, sẽ mở thẳng đến trang chứa nội dung cần tìm. Như vậy, để tìm được nội dung cần tìm, người đọc đã:
*- đọc ít hơn do mục lục rất ngắn gọn => **tìm kiếm ít hơn***
*- biết số trang chứa nội dung nên phải mở ít số trang sách hơn => **đọc ít đĩa hơn***



Index

A
About cordless telephones 51
Advanced operation 17
Answer an external call during an intercom call 15
Answering system operation 27

B
Basic operation 14
Battery 9, 38

C
Call log 22, 37
Call waiting 14
Chart of characters 18

D
Date and time 8
Delete from redial 26
Delete from the call log 24
Delete from the directory 20
Delete your announcement 32
Desk/table bracket installation 4
Dial a number from redial 26
Dial type 4, 12
Directory 17
DSL filter 5

E
Edit an entry in the directory 20
Edit handset name 11

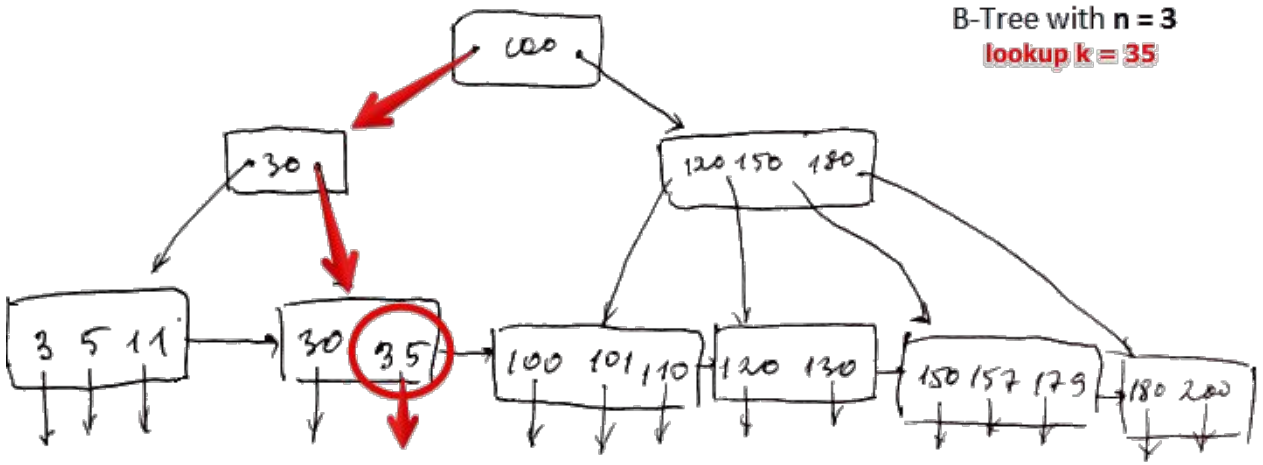
F
FCC, ACTA and IC regulations 53
Find handset 16

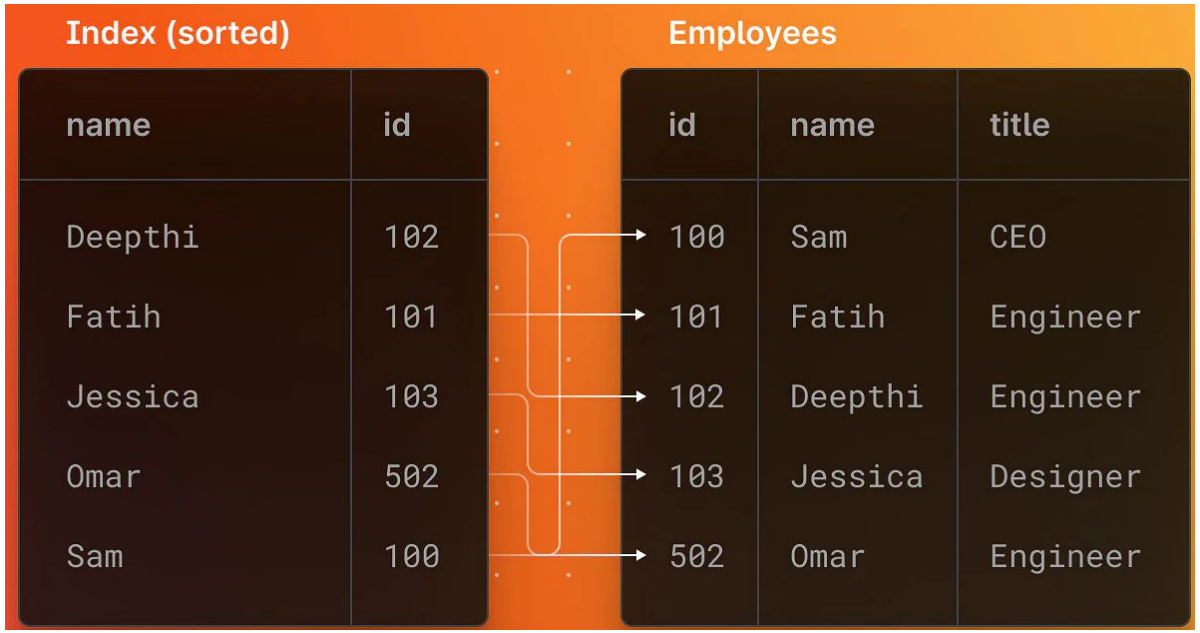
H
Handset display screen messages 36
Handset layout 6

I
Important safety instructions 39
Index 56-57
Installation 1
Install handset battery 2
Intercom call 15
Internet 4

Cách thức lưu trữ và hoạt động (B-Tree (*Balanced Tree*))

Nhờ cấu trúc lưu trữ dạng hình cây, database engine sẽ phải duyệt ít dữ liệu hơn khi tìm kiếm.
Như ví dụ cây ở hình dưới, database engine sẽ ***chỉ cần duyệt 3 lần*** *để tìm được giá trị cần (thay vì duyệt hết các giá trị)*





Về mặt lưu trữ, index sẽ được lưu trữ trên vùng ***Index Page*** *theo cấu trúc dạng hình cây, còn dữ liệu sẽ được lưu trữ trên vùng* ***Data Page.*** *Trên index sẽ lưu trữ thông tin để tham chiếu đến dữ liệu*

Ưu, nhược điểm

- Ưu điểm
  - Cải thiện hiệu năng truy vấn dữ liệu: SELECT, UPDATE, DELETE
    - *Cải thiện được hiệu năng truy vấn cho UPDATE, DELETE khi đánh index cho cột trong trong mệnh đề WHERE*
  - Giảm tỷ lệ đọc dữ liệu trên đĩa
- Nhược điểm
  - Tốn thêm dung lượng lưu trữ index (Index Page)
  - Giảm hiệu năng INSERT, DELETE (đôi khi cả UPDATE)
    - *Giảm hiệu năng của UPDATE khi cột dữ liệu được update có trong index*

Mặc dù có một số nhược điểm, nhưng hầu hết các ứng dụng đều có đặc điểm là tần suất đọc cao hơn rất nhiều so với ghi. Do vậy, nhìn chung, việc đánh index sẽ mang lại nhiều lợi ích hơn.

Vì vậy, khi thi công, hãy luôn nhớ ưu tiên việc sử dụng index để cải thiện hiệu năng truy vấn

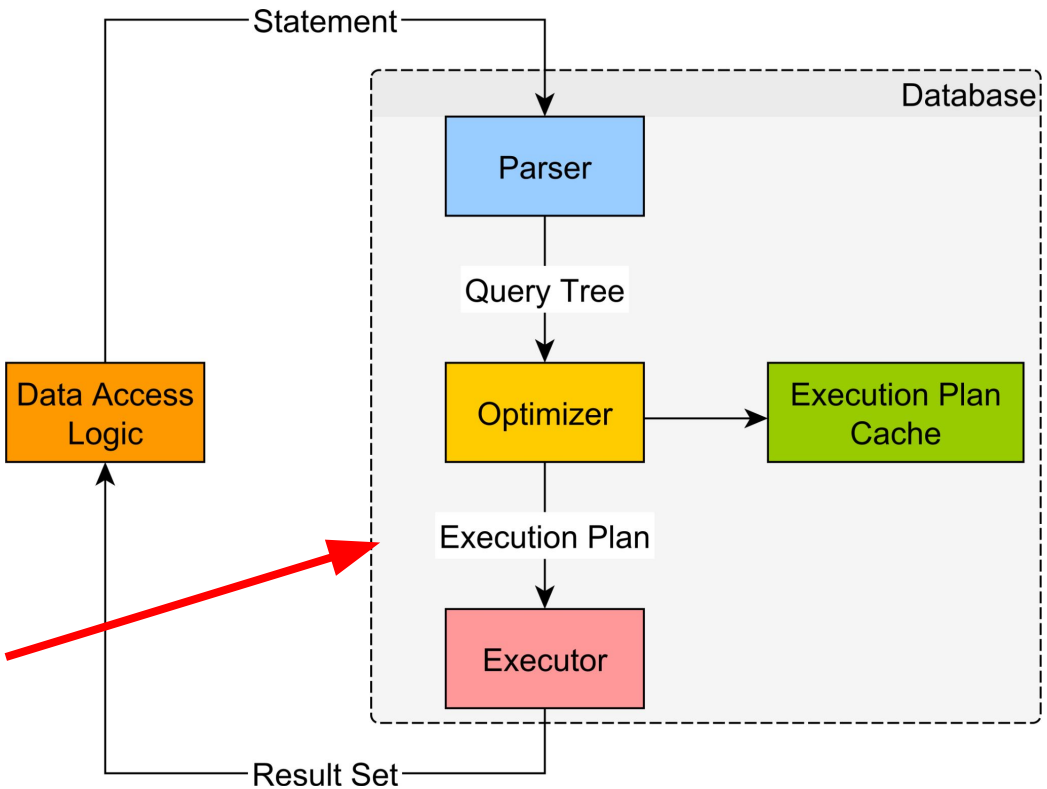
PHẦN 2

EXECUTION PLAN

Giới thiệu

- Trước khi thực hiện câu truy vấn, trình optimizer sẽ dựa vào cấu trúc bảng, index và thống kê dữ liệu trong bảng để đưa ra cách thực hiện câu truy vấn sao cho tối ưu nhất. Đó chính là Execution plan
- Execution plan chứa các thông tin mô tả cách database engine thực thi câu truy vấn, bao gồm:
  - *Cách thức truy vấn dữ liệu từ bảng: table scan, index scan hay index lookup, ……*
  - *Cách thức thực hiện sắp xếp, nhóm dữ liệu*
  - *Ước lượng chi phí thực hiện, số dòng dữ liệu trả về, …*
  - *…*

=> Những thông tin này sẽ giúp chỉ ra chậm ở đâu, chậm như thế nào, từ đó có thể áp dụng những kỹ thuật tối ưu phù hợp

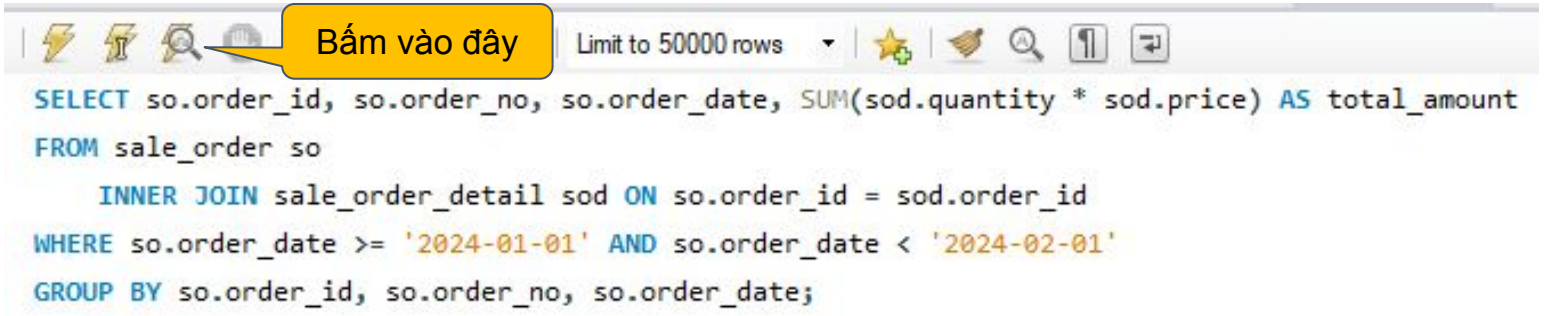


Một số khái niệm quan trọng trong Execution Plan

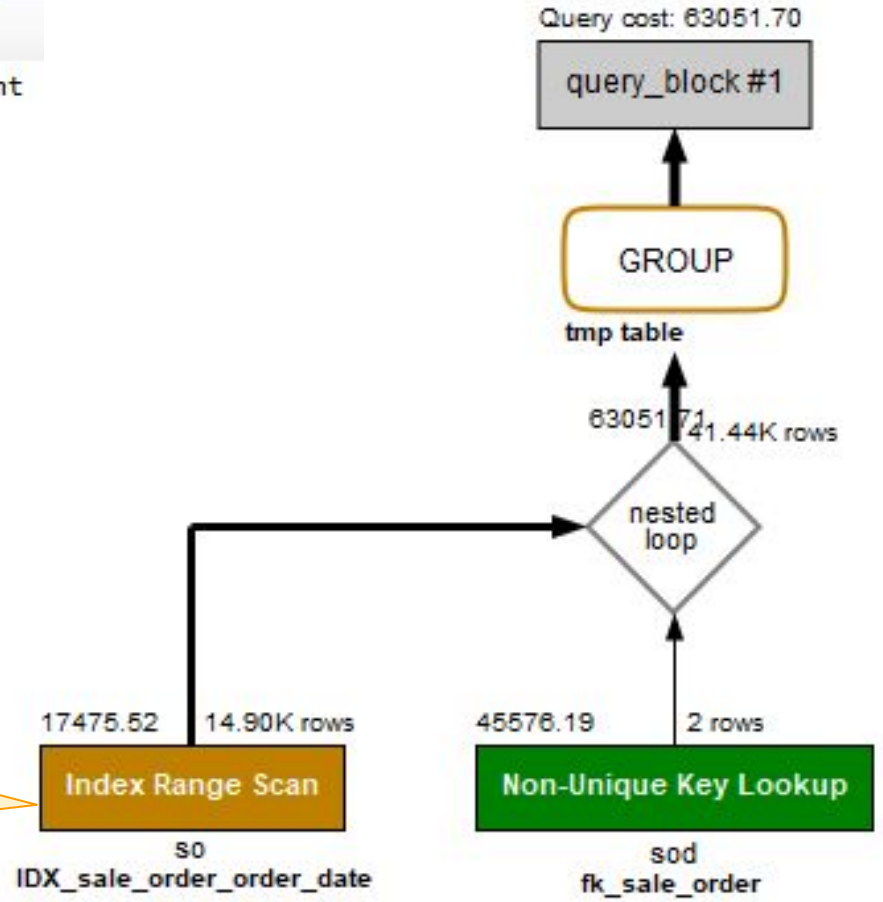
- Cách thức truy xuất dữ liệu từ bảng
  - Unique key lookup
  - Non-unique key lookup
  - Index range scan
  - Full table scan
- Thuật toán JOIN dữ liệu giữa hai bảng
  - Nested-Loop Join
  - Hash Join
- Cách thức thực hiện nhóm, sắp xếp dữ liệu
- Chi phí thực hiện, số dòng dữ liệu truy xuất

Cách dùng Execution plan trong MySql

Cách 1: Dùng Visual explain trên Workbench



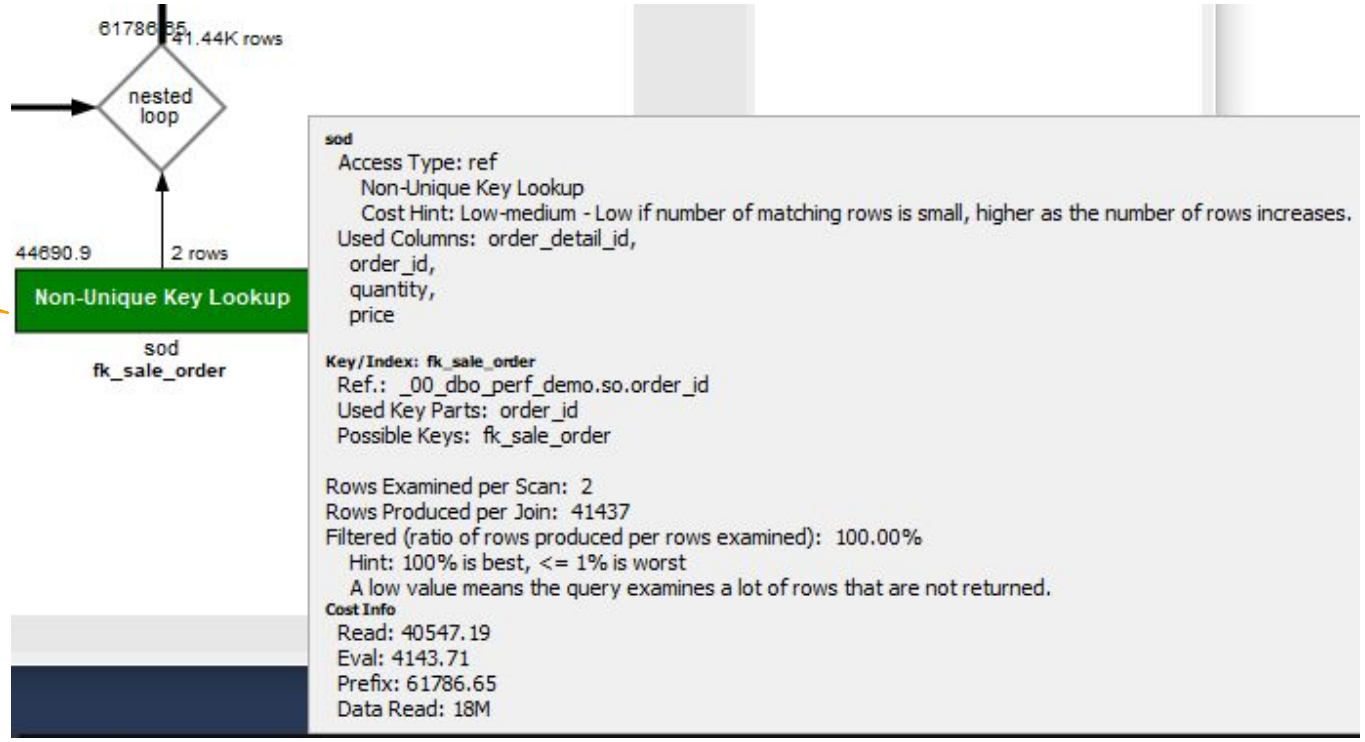
Hình chữ nhật thể hiện các thông tin về việc truy xuất dữ liệu từ bảng:
*- **Index Range Scan**: thuật toán truy xuất dữ liệu. Khi sử dụng các thuật toán tốt, hình chữ nhật sẽ có màu xanh, kém hơn chút sẽ có màu cam và tệ nhất sẽ là màu đỏ*
*- **so:** là tên bảng hoặc alias của bảng*
*- **IDX_sale_order_order_date**: là tên index sử dụng. Thông tin này chỉ xuất hiện khi truy vấn có sử dụng index*
*- **17475.52**: là chi phí để truy xuất dữ liệu. Cứ nhỏ hơn là tốt hơn*
*- **14.09K rows**: là số dòng dữ liệu cần truy xuất từ bảng*



Cách dùng Execution plan trong MySql

Cách 1: Dùng Visual explain trên Workbench

Khi hover chuột vào các hình trong Visual Explain sẽ hiển thị chi tiết hơn các thông tin.



Execution plan trong MySql

Cách 2: Dùng lệnh EXPLAIN trên SQL Editor

```
1  EXPLAIN
2  SELECT so.order_id, so.order_no, so.order_date, SUM(sod.quantity * sod.price) AS total_amount
3  FROM sale_order so
4      INNER JOIN sale_order_detail sod ON so.order_id = sod.order_id
5  WHERE so.order_date >= '2024-01-01' AND so.order_date < '2024-02-01'
6  GROUP BY so.order_id, so.order_no, so.order_date;
```

Table (read only)

id UNSIGN...	select_type VARCHAR(19)	table VARCHAR(64)	partitions NONE	type VARCHAR(10)	possible_keys VARCHAR(4096)	key VARCHAR(64)	key_len VARCHAR(4096)	ref VARCHAR(1024)	rows UNSIGNED BIGINT	filtered DOUBLE(4, 2)	Extra VARCHAR(255)
1	SIMPLE	so	*(null)*	range	PRIMARY,IDX_sale_order_order_date	IDX_sale_order_order_date	5	*(null)*	14902	100	Using index condition; Using temporary
1	SIMPLE	sod	*(null)*	ref	fk_sale_order	fk_sale_order	146	_00_dbo_perf_demo.so.order_id	2	100	*(null)*

Ngầm định, lệnh EXPLAIN sẽ trả về dữ liệu dạng bảng. Ở định dạng này sẽ cần lưu ý đến các cột sau:

- table: là tên bảng hoặc alias của bảng
- type: kiểu truy xuất dữ liệu từ bảng
  - *Một số giá trị tốt: eq_ref (Unique Lookup), ref (Non-Unique Key Lookup), ref_or_null (Key Lookup + Fetch NULL Values)*
  - *Một số giá trị tốt vừa: range (Index Range Scan)*
  - *Một số giá trị không tốt: all (Full Table Scan), index (Full Index Scan)*
- key: index được sử dụng. Nếu (null) nghĩa là không có index nào được sử dụng

Execution plan trong MySql

Cách 2: Dùng lệnh EXPLAIN trên SQL Editor

*Ngoài định dạng TABLE, TREE, lệnh EXPLAIN còn trả về định dạng **JSON** (đầu vào cho Visual Explain trên Workbench)*

```
1  EXPLAIN FORMAT=TREE
2  SELECT so.order_id, so.order_no, so.order_date, SUM(sod.quantity * sod.price) AS total_amount
3  FROM sale_order so
4      INNER JOIN sale_order_detail sod ON so.order_id = sod.order_id
5  WHERE so.order_date >= '2024-01-01' AND so.order_date < '2024-02-01'
6  GROUP BY so.order_id, so.order_no, so.order_date;
```

```
EXPLAIN
-> Table scan on <temporary>
    -> Aggregate using temporary table
        -> Nested loop inner join  (cost=63051.71 rows=41437)
            -> Index range scan on so using IDX_sale_order_order_date over ('2024-01-01 00:00:00' <= order_date < '2024-02-01 00:00:00'), with index condition: (
(so.order_date >= TIMESTAMP'2024-01-01 00:00:00') and (so.order_date < TIMESTAMP'2024-02-01 00:00:00'))  (cost=17475.52 rows=14902)
            -> Index lookup on sod using fk_sale_order (order_id=so.order_id)  (cost=2.78 rows=3)
```

*Với định dạng **TREE**, lệnh **EXPLAIN** sẽ trả về text dạng tree. Mỗi dòng thường sẽ gồm các thông tin như sau:*
- ***Index lookup**: thuật toán truy xuất dữ liệu từ bảng*
- ***sod**: tên bảng hoặc alias*
- ***fk_sale_order**: tên index sử dụng*
- ***cost=2.78**: chi phí ước lượng để thực hiện truy xuất dữ liệu từ bảng*
- ***rows=3**: là số dòng dữ liệu từ bảng sau khi lọc*

Execution plan trong MySql

Lưu ý: Lệnh **EXPLAIN** không thực sự thực thi câu truy vấn. Khi đó, các thông tin trên Execution plan là các thông tin do MySql ước lượng.
Để thực thi câu truy vấn và lấy các thông tin thực tế, dùng lệnh **EXPLAIN ANALYZE**

```
1  EXPLAIN ANALYZE
2  SELECT so.order_id, so.order_no, so.order_date, SUM(sod.quantity * sod.price) AS total_amount
3  FROM sale_order so
4      INNER JOIN sale_order_detail sod ON so.order_id = sod.order_id
5  WHERE so.order_date >= '2024-01-01' AND so.order_date < '2024-02-01'
6  GROUP BY so.order_id, so.order_no, so.order_date;
```

```
EXPLAIN
-> Table scan on <temporary>  (actual time=2419.238..2420.163 rows=7107 loops=1)
    -> Aggregate using temporary table  (actual time=2419.236..2419.236 rows=7107 loops=1)
        -> Nested loop inner join  (cost=63051.71 rows=41437) (actual time=7.425..2326.952 rows=19171 loops=1)
            -> Index range scan on so using IDX_sale_order_order_date over ('2024-01-01 00:00:00' <= order_date < '2024-0      0:00:00'), with index condition: (
(so.order_date >= TIMESTAMP'2024-01-01 00:00:00') and (so.order_date < TIMESTAMP'2024-02-01 00:00:00'))  (cost=17475.52 ro    14902) (actual time=1.527..594.161
rows=7734 loops=1)
            -> Index lookup on sod using fk_sale_order (order_id=so.order_id)  (cost=2.78 rows=3) (actual time=0.211..0.223 rows=2 loops=7734)
```

Thông tin thực tế khi chạy câu truy vấn:
- Thời gian lấy row đầu tiên (0.211)
- Thời gian lấy tất cả các rows (0.223)
- Số lượng rows thực tế (2)
- Thời gian lặp lại việc truy xuất dữ liệu từ bảng sod là 7734 lần

Đọc thêm chi tiết về execution plan: https://planetscale.com/blog/how-read-mysql-explains

Execution plan trong MySql

- Một số dấu hiệu chỉ báo các điểm cần cải tiến
  - Thuật toán truy xuất dữ liệu là Full Table Scan, Full Index Scan
  - Số dòng dữ liệu lớn
  - Chi phí lớn

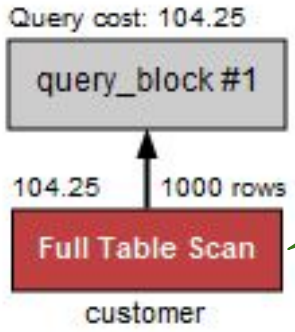
PHẦN 3

THIẾT KẾ DATABASE ĐẢM BẢO HIỆU NĂNG

Thiết kế index

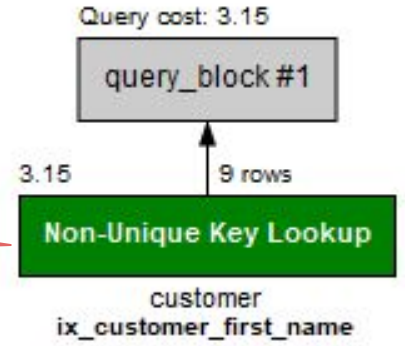
1. Chọn cột xuất hiện trong mệnh đề WHERE

```
SELECT first_name, last_name, address
FROM customer
WHERE first_name = "Jimmy"
```



*Khi chưa sử dụng index, câu truy vấn sẽ dùng thuật toán "**Full Table Scan**" và scan **1000 rows** với **cost 104.25***

```
CREATE INDEX ix_cutsomer_first_name ON customer(first_name)
```



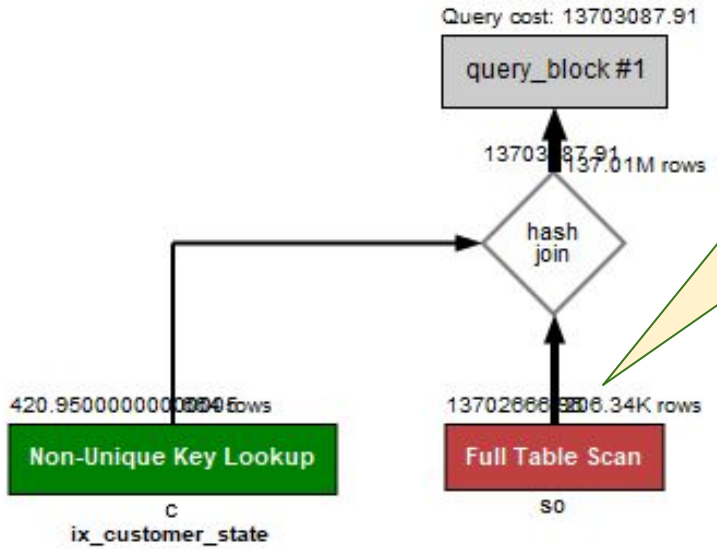
*Khi bổ sung index, câu truy vấn sẽ dùng thuật toán "**Non-Unique Key Lookup**" và scan **9 rows** với **cost 3.15***

Thiết kế index

2. Chọn cột xuất hiện trong mệnh đề JOIN

```
SELECT first_name, last_name, address
FROM customer c LEFT JOIN sale_order so on c.customer_id = so.customer_id
WHERE c.state = 'Texas'
```

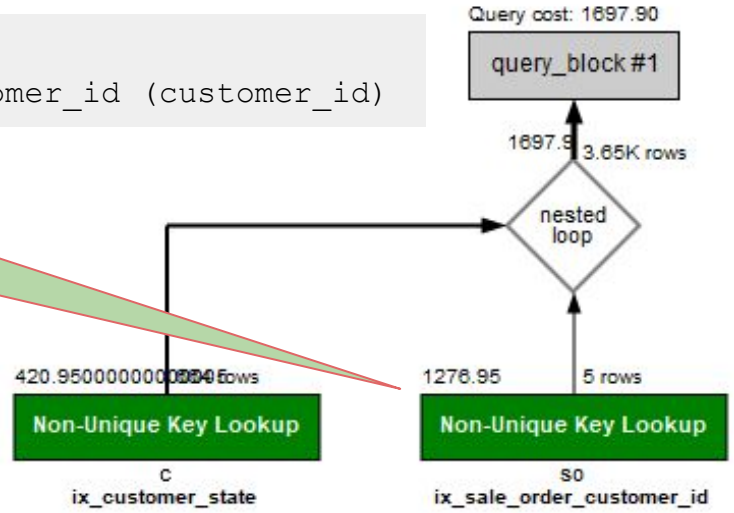
*Khi chưa sử dụng index cho cột customer_id trên bảng sale_order, câu truy vấn sẽ dùng thuật toán "**Full Table Scan**" và scan **218.85k rows** với **cost 14539758.25***



Cost tổng thể cũng giảm rất nhiều

```
ALTER TABLE sale_order
ADD INDEX ix_sale_order_customer_id (customer_id)
```

*Khi bổ sung index, câu truy vấn sẽ dùng thuật toán "**Non-Unique Key Lookup**" và **cost chỉ còn 1276.95***



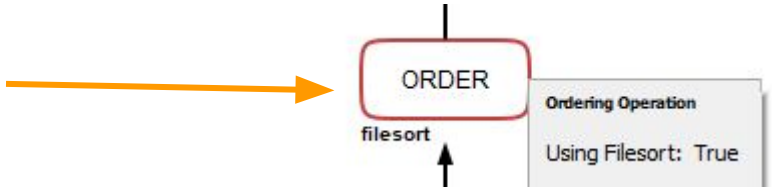
Thiết kế index

3. Chọn cột xuất hiện trong mệnh đề ORDER BY

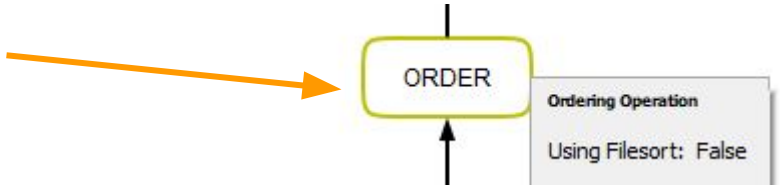
Mệnh đề ORDER BY sẽ chậm khi phải sử dụng filesort (đọc và sắp xếp lại dữ liệu từ bảng) với lượng dữ liệu lớn. Để cải thiện hiệu năng, sẽ cần bổ sung index để phù hợp với câu truy vấn.

Khi sử dụng Visual Explain:

- ORDER BY có sử dụng filesort sẽ được thể hiện bởi hình chữ nhật viền màu đỏ



- ORDER không sử dụng filesort sẽ được thể hiện bởi hình chữ nhật viền màu vàng



Thiết kế index

3. Chọn cột xuất hiện trong mệnh đề ORDER BY

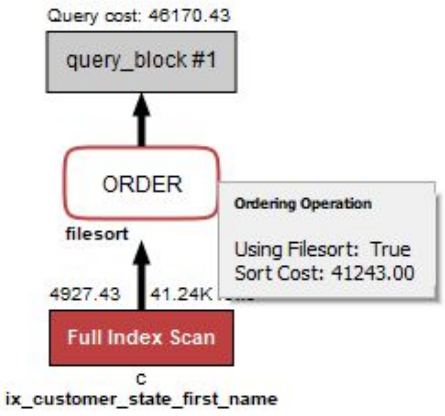
Giả sử có index như sau

```
ALTER TABLE customer ADD INDEX ix_customer_state_first_name (state, first_name)
```

```
SELECT first_name
FROM customer c
ORDER BY first_name;
```

ain | Display Info: Read + Eval cost | Overview:

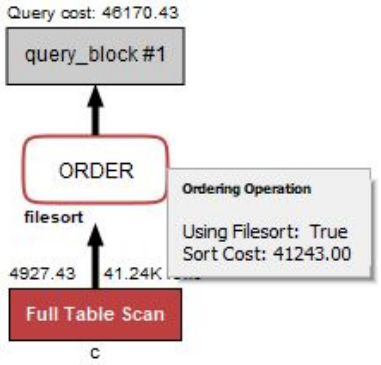
*Không sử dụng được index do cột trong ORDER BY **không phải là cột đứng trước trong index***




```
SELECT first_name, last_name
FROM customer c
ORDER BY state, first_name;
```

lain | Display Info: Read + Eval cost | Overview:

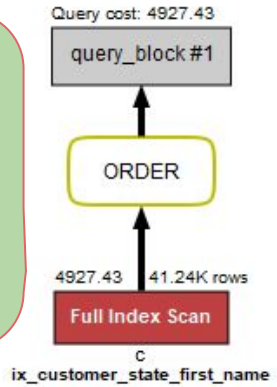
*Không sử dụng được index do có **SELECT cột không có trong index** (cột last_name). MySql đánh giá lấy dữ liệu từ bảng sẽ hiệu quả hơn*




```
select first_name
from customer c
order by state, first_name;
```

qplain | Display Info: Read + Eval cost |

Trường hợp sử dụng index hiệu quả:
- Chỉ select cột trong index
- ORDER BY theo đúng thứ tự các cột trong index



Thiết kế index

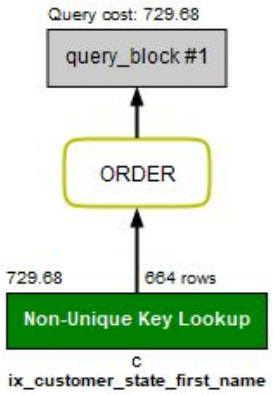
3. Chọn cột xuất hiện trong mệnh đề ORDER BY

Giả sử có index như sau

```
ALTER TABLE customer ADD INDEX ix_customer_state_first_name (state, first_name)
```

```
SELECT first_name, last_name
FROM customer c
WHERE state = 'Texas'
ORDER BY first_name;
```

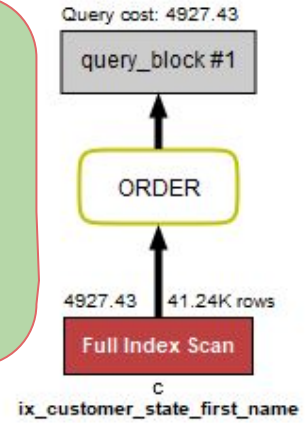
xplain | Display Info: Read + Eval cost



*Mặc dù có SELECT cột không nằm trong index và cột trong ORDER BY không đứng trước trong index, nhưng câu truy vấn này vẫn dùng index hiệu quả là nhờ **kết hợp với điều kiện WHERE** đã lọc dữ liệu. Phần dữ liệu sau khi lọc chỉ còn "state = 'Texas'"*

```
SELECT first_name
FROM customer c
ORDER BY state DESC, first_name DESC;
```

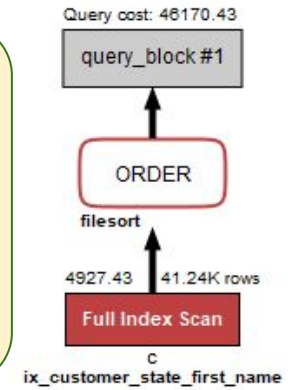
plain | Display Info: Read + Eval cost



*Các cột trong ORDER BY có hướng ngược với các cột trong index. Tuy nhiên, **tất cả các cột đều ngược (hướng đồng nhất)** nên câu truy vấn vẫn sử dụng index hiệu quả*

```
SELECT first_name
FROM customer c
ORDER BY state ASC, first_name DESC;
```

plain | Display Info: Read + Eval cost



*Các cột trong ORDER BY có **hướng không đồng nhất** với các cột trong index nên câu truy vấn không sử dụng được index*

Thiết kế index

- Chọn cột xuất hiện trong mệnh đề WHERE, JOIN, ORDER BY

  Để chọn được những cột đánh index hiệu quả, đòi hỏi người thiết kế phải hiểu được dữ liệu lưu trữ trong bảng. Khi thiết kế, phải đánh giá được dữ liệu trong bảng sẽ được truy vấn như thế nào, điều kiện lọc dữ liệu là gì?

  Một số cột điển hình thường chọn làm index
  - tenant_id: xuất hiện trong mô hình database POOL
  - organization_id: đáp ứng câu truy vấn lấy dữ liệu theo đơn vị
  - Cột kiểu date như ngày chứng từ, ngày ghi sổ: lấy dữ liệu báo cáo
  - Foreign key: đáp ứng JOIN Master-Detail, hoặc lấy dữ liệu trong hàm GetDetailByMasterId

Thiết kế index

4. Sử dụng Composite index hay Single column Index

Composite index là index gồm nhiều cột. Composite Index thường được sử dụng cho những cột thường đứng cùng nhau trong các mệnh đề WHERE, JOIN, ORDER BY. Khi đó, loại index này sẽ mang lại hiệu quả truy vấn tốt hơn so với Single column index

```
1  EXPLAIN ANALYZE
2  SELECT so.order_id, so.order_no, so.order_date
3  FROM sale_order so
4  WHERE so.customer_id = '11116d78-73fb-72a8-8cd3-66948daf6128'
5        AND so.order_date >= '2024-01-01' AND so.order_date < '2024-02-01';
6
```

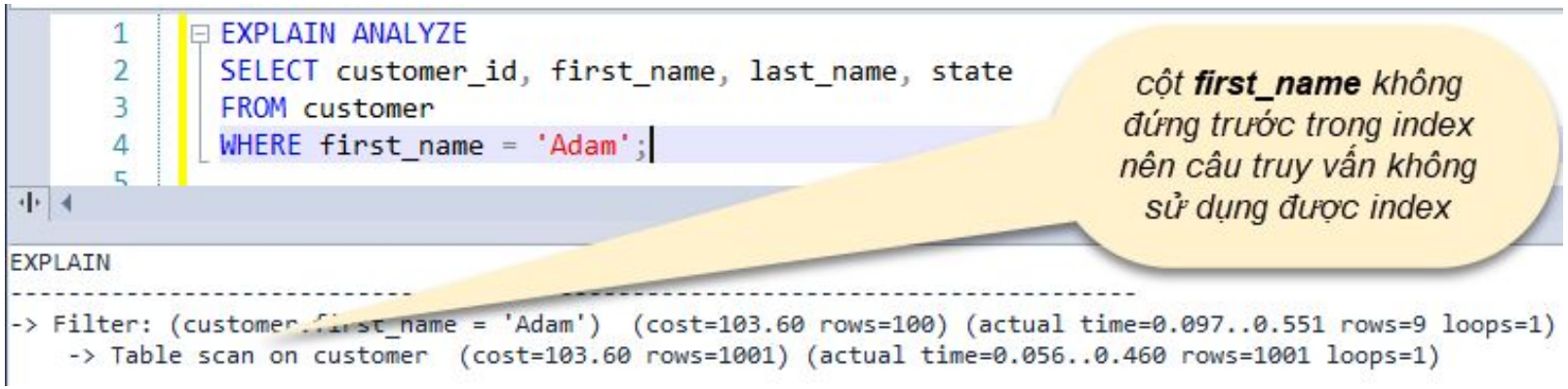
```
EXPLAIN
-----------------------
```

Thiết kế index

4. Sử dụng Composite index hay Single column Index

- Thứ tự cột trong index
  - *Câu truy vấn chỉ sử dụng index khi các cột từ trái qua trong index xuất hiện trong mệnh đề WHERE, JOIN*

```
ALTER TABLE customer ADD INDEX ix_customer_state_first_name (state, first_name)
```

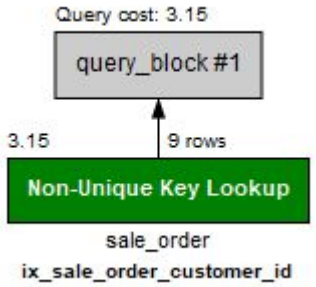


Thiết kế index

4. Sử dụng Composite index hay Single column Index

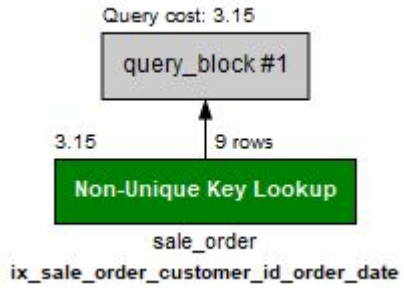
- Tránh dư thừa index
  - *Việc dư thừa index xảy ra khi trên một bảng có nhiều hơn 1 index chứa các cột giống nhau hoặc có index có các cột theo thứ từ trái qua là tập con của một index khác.*
    Ví dụ có 2 index:

```
CREATE INDEX ix_sale_order_customer_id_order_date ON sale_order(customer_id, order_date)
CREATE INDEX ix_sale_order_customer_id ON sale_order(customer_id) # => index này thừa
```




```
SELECT order_id, order_no, order_date, customer_id
FROM sale_order
WHERE customer_id = '119af01c-406e-1565-a77d-e8c757976496';
```

Dù dùng index nào thì cost cũng tương đương



Thiết kế index

4. Sử dụng Composite index hay Single column Index

- Chọn cột cho Composite index

  Composite index gồm nhiều cột. Các bảng dữ liệu chính của ứng dụng thường có nhiều dạng yêu cầu truy vấn dữ liệu, có vấn đề đặt ra là chọn và sắp xếp các cột như thế nào để Composite index có thể đáp ứng được cho nhiều câu truy vấn nhất có thể? Có một số gợi ý như sau:

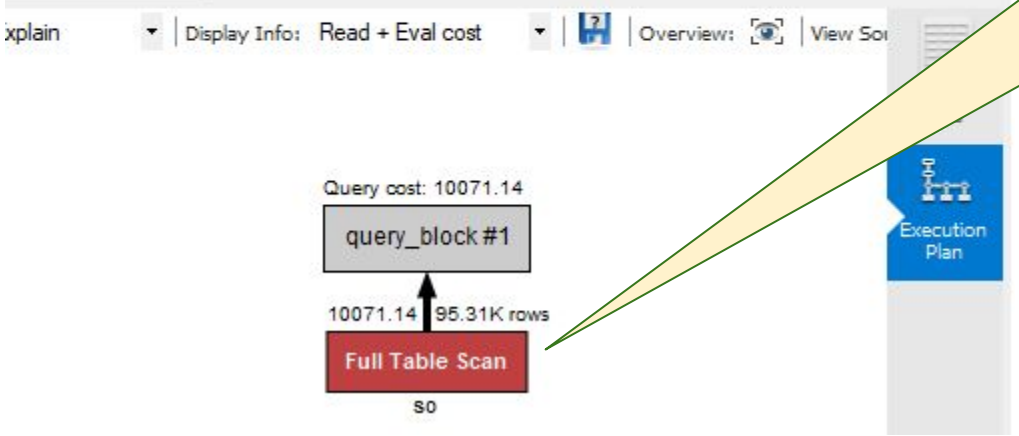
  - *Ưu tiên cột đứng trước là những cột thường xuất hiện nhiều nhất trong các câu truy vấn*
    - *VD: Có nhiều câu truy vấn lấy dữ liệu bảng order đều có điều kiện lọc theo customer_id => nên ưu tiên cột customer_id đứng trước để đảm bảo index đáp ứng được nhiều câu truy vấn*
  - *Ưu tiên cột đứng trước là các cột thường dùng toán tử =, IN khi truy vấn*
    - *VD: khi câu truy vấn có điều kiện: "customer_id = ? AND order_date > '2024-01-01'" => nên bổ sung index với cột customer_id đứng trước*
  - *Ưu tiên cột đứng trước là các cột chứa nhiều giá trị đơn nhất (có COUNT(DISTINCT column_name) lớn hơn)*

PHẦN 4

THI CÔNG DATABASE ĐẢM BẢO HIỆU NĂNG

Tránh sử dụng Function với cột có INDEX trong WHERE

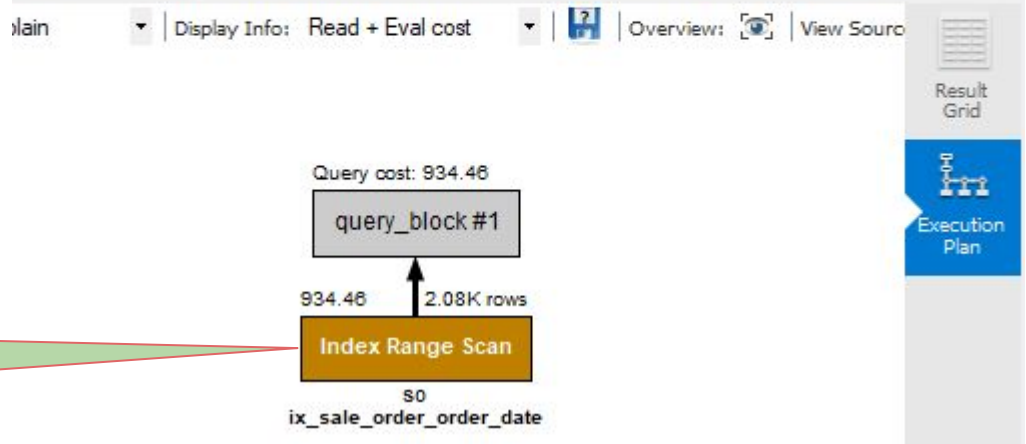
```sql
SELECT so.order_id, so.order_no, so.order_date, total_amount
FROM sale_order so
WHERE YEAR(so.order_date) = 2024 AND MONTH(so.order_date) = 1;
```



*Khi sử dụng Function với cột được đánh index, mysql sẽ không tận dụng được index và sẽ sử dụng thuật toán "**Full Table Scan**", do đó chi phí thực hiện câu truy vấn sẽ cao*

Khi đó, cần tìm cách biến đổi để câu truy vấn có thể sử dụng được index

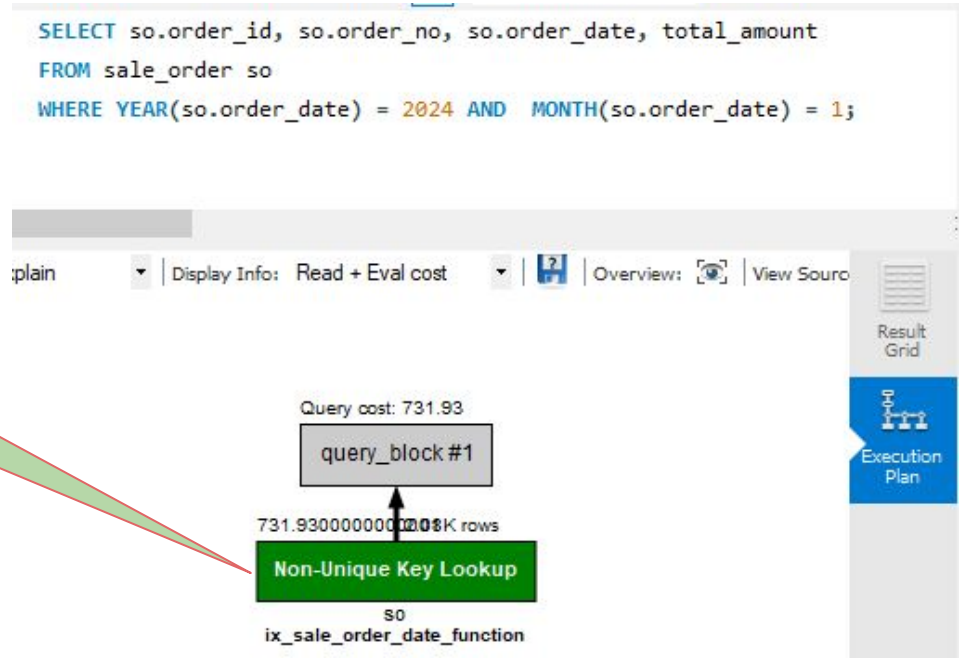
```sql
SELECT so.order_id, so.order_no, so.order_date, total_amount
FROM sale_order so
WHERE so.order_date >= '2024-01-01' AND so.order_date < '2024-02-01';
```



Tránh sử dụng Function với cột có INDEX trong WHERE

```
CREATE INDEX ix_sale_order_order_function ON sale_order((YEAR(order_date)),(MONTH(order_date)));
```

*Một giải pháp nữa để cải tiến khi phải sử dụng Function với cột trong WHERE là sử dụng **Functional Index**. Khi đó, câu truy vấn sẽ sử dụng index để tăng hiệu năng*



Gợi ý:
Trong 2 phương án cải tiến thì cần thử nghiệm xem phương án nào hiệu quả hơn để lựa chọn. Trường hợp mang lại hiệu quả như nhau thì nên ưu tiên phương án biến đổi để sử dụng index.

Tránh sử dụng SELECT DISTINCT (tương tự với UNION)

Khi có DISTINCT, MySql phải làm thêm việc

```
1  EXPLAIN ANALYZE
2  SELECT DISTINCT so.order_no, so.order_date, total_amount
3  FROM sale_order so
4  WHERE so.order_date >= '2024-01-01' AND so.order_date < '2024-04-01';
5
6  EXPLAIN ANALYZE
7  SELECT so.order_no, so.order_date, total_amount
8  FROM sale_order so
9  WHERE so.order_date >= '2024-01-01' AND so.order_date < '2024-04-01';
```

```
EXPLAIN
-----------------------------------------------------------------------------
-> Table scan on <temporary>  (cost=0.01..554.50 rows=44160) (actual time=174.503..177.742 rows=22551 loops=1)
    -> Temporary table with deduplication  (cost=24288.27..24842.76 rows=44160) (actual time=174.499..174.499 rows=22551 loops=1)
        -> Index range scan on so using ix_sale_order_order_date over ('2024-01-01 00:00:00' <= order_date < '2024-04-01 00:00:00'), with index condition: ((so.
order_date >= TIMESTAMP'2024-01-01 00:00:00') and (so.order_date < TIMESTAMP'2024-04-01 00:00:00'))  (cost=19872.26 rows=44160) (actual time=0.065..118.677 rows=
22551 loops=1)


1 records fetched

EXPLAIN
-----------------------------------------------------------------------------
-> Index range scan on so using ix_sale_order_order_date over ('2024-01-01 00:00:00' <= order_date < '2024-04-01 00:00:00'), with index condition: ((so.order_date >
= TIMESTAMP'2024-01-01 00:00:00') and (so.order_date < TIMESTAMP'2024-04-01 00:00:00'))  (cost=19872.26 rows=44160) (actual time=0.036..105.008 rows=22551 loops=1)
```

Khi sử dụng DISTINCT, chi phí để thực hiện câu truy vấn sẽ lớn hơn nhiều. Vì vậy, hãy xem xét nghiệp vụ có thể bỏ DISTINCT không? Nếu không phải tìm cách tối ưu

Như trường hợp ví dụ trên, các giá trị trong cột order_no trong bảng sale_order là duy nhất.
Do vậy, sẽ không cần sử dụng DISTINCT

Tránh sử dụng SELECT DISTINCT (tương tự với UNION)

- Một số lỗi thường gặp khi sử dụng DISTINCT và UNION
  - Nghiệp vụ không yêu cầu nhưng vẫn sử dụng DISTINCT, UNION
  - Sử dụng DISTINCT nhưng có cột UNIQUE trong SELECT
  - Sử dụng UNION để hợp nhất dữ liệu từ 2 bảng, và dữ liệu từ 2 bảng đó chắc chắn khác nhau

Việc sử dụng DISTINCT, UNION cần căn cứ vào yêu cầu nghiệp vụ.

Tuy nhiên, hãy luôn ghi nhớ là hai mệnh đề này tốn kém chi phí, và có hiệu năng thấp. Do đó, hãy luôn xem xét thật kỹ xem có cần thiết sử dụng không? Và nếu cần thì hãy thử các phương pháp khử DISTINCT, UNION đi để cải thiện hiệu năng

Một ví dụ về sử dụng sub-query để khử DISTINCT

```sql
SELECT DISTINCT c.customer_id, c.first_name, c.last_name, c.address
FROM customer c
    INNER JOIN sale_order so ON c.customer_id = so.customer_id
WHERE so.order_date >= '2024-01-01' AND so.order_date < '2024-04-01';

-- Khử DISTINCT
SELECT c.customer_id, c.first_name, c.last_name, c.address
FROM customer c
WHERE c.customer_id IN (SELECT so.customer_id FROM sale_order so
                        WHERE so.order_date >= '2024-01-01'
                              AND so.order_date < '2024-04-01');
```

Chỉ trả về dữ liệu vừa đủ dùng

- Sử dụng LIMIT để giới hạn số dòng trả về
- Không sử dụng SELECT *, hãy luôn SELECT đúng các cột cần thiết

Khi sử dụng LIMIT, và chỉ SELECT các cột cần thiết, MySql sẽ
- đọc đĩa ít hơn => giảm Disk I/O
- trả về ít dữ liệu hơn => giảm Network I/O

```
localhost:33060+  Py > #QPS
localhost:33060+  Py > import time
t:33060+  Py > query = "SELECT order_id, order_no, order_date FROM sale_order;"
33060+  Py > nb_of_requests = 200
33060+  Py > start_time = time.time()
33060+  Py > for x in range(0, nb_of_requests):
             ->   sql = query
             ->   result = session.run_sql(sql)
             -> total_time = time.time() - start_time
             -> qps = nb_of_requests / total_time
             -> print(" - Total time: %s seconds -" % total_time)
             -> print(" --- %s QPS ---" % qps)
             ->
- Total time: 16.98180365562439 seconds -
--- 11.777312001470456 QPS ---
MySQL  localhost:33060+  Py > #QPS
MySQL  localhost:33060+  Py > import time
MySQL  localhost:33060+  Py > query = "SELECT order_id, order_no, order_date, total_amount, status, customer_id FROM sale_order;"
MySQL  localhost:33060+  Py > nb_of_requests = 200
MySQL  localhost:33060+  Py > start_time = time.time()
MySQL  localhost:33060+  Py > for x in range(0, nb_of_requests):
             ->   sql = query
             ->   result = session.run_sql(sql)
             -> total_time = time.time() - start_time
             -> qps = nb_of_requests / total_time
             -> print(" - Total time: %s seconds -" % total_time)
             -> print(" --- %s QPS ---" % qps)
             ->
- Total time: 23.000059843063354 seconds -
--- 8.695629548995218 QPS ---
```

Khi SELECT 3 cột, chạy liên tục 200 lần thì duration là 16.9 giây

Khi SELECT 6 cột thì duration tăng lên 23 giây

Sử dụng SET-based thay vì ROW-based

- Khái niệm ROW-based dùng để ám chỉ việc Database Engine thực thi câu truy vấn bằng cách xử lý tính toán trên từng dòng dữ liệu
- SET-based ám chỉ việc Database Engine thực thi câu truy vấn bằng cách xử lý tính toán trên một tập hợp các dòng dữ liệu
- Các câu truy vấn dùng SET-based thường có hiệu năng cao hơn ROW-based rất nhiều lần

Sử dụng SET-based thay vì ROW-based

- Ví dụ điển hình về ROW-based là sử dụng **CURSOR**

```
PROCEDURE proc_sale_get_revenue_summary_use_cursor()
BEGIN
    DECLARE v_done INT DEFAULT 0;
    DECLARE v_customer_id varchar(36);
    DECLARE v_first_name varchar(50);
    DECLARE v_last_name VARCHAR(50);
    DECLARE v_total_amount decimal(18, 4);

    DECLARE cur CURSOR FOR SELECT customer_id, first_name, last_name FROM customer;
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_done = 1;

    -- Create a temporary table to hold the results
    DROP TEMPORARY TABLE IF EXISTS temp_revenue_summaries;
    CREATE TEMPORARY TABLE temp_revenue_summaries (
        customer_id varchar(36), first_name VARCHAR(50), last_name VARCHAR(50), total_amount DECIMAL(18, 4));

    OPEN cur;
    read_loop: LOOP
        FETCH cur INTO v_customer_id, v_first_name, v_last_name;
        IF v_done THEN
            LEAVE read_loop;
        END IF;

        SELECT SUM(total_amount) INTO v_total_amount
        FROM sale_order
        WHERE customer_id = v_customer_id;

        -- Insert the result into the temporary table
        INSERT INTO temp_revenue_summaries (customer_id, first_name, last_name, total_amount)
        VALUES (v_customer_id, v_first_name, v_last_name, v_total_amount);
    END LOOP;
    CLOSE cur;

    SELECT customer_id, first_name, last_name, total_amount FROM temp_revenue_summaries;
    DROP TEMPORARY TABLE IF EXISTS temp_revenue_summaries;
END
```

*Ví dụ về việc sử dụng **CURSOR***
Ví dụ này có thể được viết lại đơn giản hơn rất nhiều như hình bên dưới

Và kết quả hiệu năng cải thiện rõ rệt (xem slide sau)

Store dùng cursor có thể được viết lại đơn giản như sau để chuyển sang sử dụng SET-based

```
PROCEDURE proc_sale_get_revenue_summaries()
BEGIN
    SELECT c.customer_id, c.first_name, c.last_name, so.sum_total_amount AS total_amount
    FROM customer c
        LEFT JOIN (SELECT customer_id, SUM(total_amount) AS sum_total_amount
                   FROM sale_order
                   GROUP BY customer_id) so ON c.customer_id = so.customer_id;
END
```

Sử dụng SET-based thay vì ROW-based

- Ví dụ điển hình về ROW-based là sử dụng **CURSOR**

```
MySQL  localhost:33060+  Py > # Sample to use CURRSOR
MySQL  localhost:33060+  Py > import time
MySQL  localhost:33060+  Py > query = """
                           -> CALL proc_sale_get_revenue_summary_use_cursor();
                           -> """
MySQL  localhost:33060+  Py > nb_of_requests = 10
MySQL  localhost:33060+  Py > start_time = time.time()
MySQL  localhost:33060+  Py > for x in range(0, nb_of_requests):
                           ->   sql = query
                           ->   result = session.run_sql(sql)
                           -> total_time = time.time() - start_time
                           -> qps = nb_of_requests / total_time
                           -> print(" - Total time: %s seconds -" % total_time)
                           -> print(" --- %s QPS ---" % qps)
                           ->
- Total time: 56.715747356414795 seconds -
--- 0.17631787406692714 QPS ---
```

Khi sử dụng CURSOR, thời gian để thực thi store 10 lần là: ***56 giây.***
Còn khi khử CURSOR, thời gian thực thi ***chỉ còn 13 giây***

```
MySQL  localhost:33060+  Py > import time
MySQL  localhost:33060+  Py > query = """
                           -> CALL proc_sale_get_revenue_summaries();
                           -> """
MySQL  localhost:33060+  Py > nb_of_requests = 10
MySQL  localhost:33060+  Py > start_time = time.time()
MySQL  localhost:33060+  Py > for x in range(0, nb_of_requests):
                           ->   sql = query
                           ->   result = session.run_sql(sql)
                           -> total_time = time.time() - start_time
                           -> qps = nb_of_requests / total_time
                           -> print(" - Total time: %s seconds -" % total_time)
                           -> print(" --- %s QPS ---" % qps)
                           ->
- Total time: 13.017825365066528 seconds -
--- 0.7681774581824631 QPS ---
```

Sử dụng SET-based thay vì ROW-based

- Ví dụ điển hình về ROW-based là sử dụng **CURSOR**

```
DECLARE cur CURSOR FOR SELECT product_id, unit_price FROM tmp_product_prices;
OPEN cur;
read_loop: LOOP
    FETCH cur INTO v_product_id, v_unit_price;
    IF v_done THEN
        LEAVE read_loop;
    END IF;

    UPDATE product
    SET unit_price = v_unit_price
    WHERE product_id = v_product_id;
END LOOP;
CLOSE cur;
```



```
UPDATE product p
      INNER JOIN tmp_product_prices tmp_p ON p.product_id = tmp_p.product_id
SET p.price = tmp_p.unit_price;
```

Một ví dụ khác về việc khử CURSOR:
*Giả sử bảng tạm **tmp_product_prices** chứa danh sách các sản phẩm cần cập nhật đơn giá. Khi đó, thay vì dùng **CURSOR duyệt qua từng sản phẩm** để cập nhật đơn giá cho bảng **product,** ta có thể **sử dụng cú pháp UPDATE** <table> **JOIN … SET …***
*Khi đó, **MySql sẽ sử dụng SET-based** để thực thi câu truy vấn*

Sử dụng SET-based thay vì ROW-based

- Ví dụ sử dụng ROW-based tiếp theo là việc gọi hàm trong câu truy vấn

```
1  EXPLAIN ANALYZE
2  SELECT so.order_id, so.order_no, so.order_date, so.total_amount
3  FROM sale_order so
4  WHERE so.order_date >= DATE_ADD(SYSDATE() , INTERVAL -30 DAY);
```

```
EXPLAIN
-----------------------------------------------------------------------------
-> Filter: (so.order_date >= (sysdate() + interval <cache>(-(30)) day))  (cost=9406.95 rows=66904) (
actual time=3.807..973.662 rows=7047 loops=1)
    -> Table scan on so  (cost=9406.95 rows=200732) (actual time=3.772..759.976 rows=200000 loops=1)
```

*Mặc dù bảng **sale_order** có index cột **order_date**, tuy nhiên, do hàm SYSDATE() trả về giá trị không cố định nên MySql sẽ phải tính lại SYSDATE() cho từng dòng dữ liệu -> Xử lý kiểu **ROW-based***

Viết lại câu truy vấn trên một chút, lấy trước giá trị SYSDATE(), khi đó, MySql sẽ sử dụng index khi thực thi.
Performance cải thiện đáng kể (xem slide sau)

```
1  set @current_date = SYSDATE();
2  EXPLAIN ANALYZE
3  SELECT so.order_id, so.order_no, so.order_date, so.total_amount
4  FROM sale_order so
5  WHERE so.order_date >= DATE_ADD(@current_date , INTERVAL -30 DAY);
```

```
EXPLAIN
-----------------------------------------------------------------------------
-> Index range scan on so using IDX_sale_order_order_date over ('2024-06-24 14:49:50' <= order_date),
 with index condition: (so.order_date >= <cache>((@`current_date`) + interval -(30) day)))  (cost=
9700.41 rows=14466) (actual time=0.129..328.622 rows=7044 loops=1)
```

Sử dụng SET-based thay vì ROW-based

- Ví dụ sử dụng ROW-based tiếp theo là việc gọi hàm trong câu truy vấn

```
MySQL  localhost:33060+  Py > #QPS
MySQL  localhost:33060+  Py > import time
MySQL  localhost:33060+  Py > query = """
                         -> SELECT so.order_id, so.order_no, so.order_date, so.total_amount
                         -> FROM sale_order so
                         -> WHERE so.order_date >= DATE_ADD(SYSDATE() , INTERVAL -30 DAY);
                         -> """
MySQL  localhost:33060+  Py > nb_of_requests = 100
MySQL  localhost:33060+  Py > start_time = time.time()
MySQL  localhost:33060+  Py > for x in range(0, nb_of_requests):
                         ->   sql = query
                         ->   result = session.run_sql(sql)
                         -> total_time = time.time() - start_time
                         -> qps = nb_of_requests / total_time
                         -> print(" - Total time: %s seconds -" % total_time)
                         -> print(" --- %s QPS ---" % qps)
                         ->
- Total time: 34.6038818359375 seconds -
--- 2.8898491930505337 QPS ---
```

*Khi trong mệnh đề WHERE có so sánh với hàm trả về **giá trị không cố định, MySql phải tính toán trên từng dòng (ROW-based) -> Mất 34 giây** để thực thi 100 lần câu truy vấn*

*Một số hàm trả về các giá trị không cố định nữa gồm: **UUID(), RAND()**, …và hàm do **người dùng định nghĩa** (có thể)*

*Khi so sánh với 1 giá trị cố định, tốc độ cải thiện đáng kể. **Chỉ cần 10 giây** để thực thi xong*

```
MySQL  localhost:33060+  Py > #QPS
MySQL  localhost:33060+  Py > import time
MySQL  localhost:33060+  Py > query = """
                         -> SELECT so.order_id, so.order_no, so.order_date, so.total_amount
                         -> FROM sale_order so
                         -> WHERE so.order_date >= DATE_ADD('2024-07-24 14:53:20' , INTERVAL -30 DAY);
                         -> """
MySQL  localhost:33060+  Py > nb_of_requests = 100
MySQL  localhost:33060+  Py > start_time = time.time()
MySQL  localhost:33060+  Py > for x in range(0, nb_of_requests):
                         ->   sql = query
                         ->   result = session.run_sql(sql)
                         -> total_time = time.time() - start_time
                         -> qps = nb_of_requests / total_time
                         -> print(" - Total time: %s seconds -" % total_time)
                         -> print(" --- %s QPS ---" % qps)
                         ->
- Total time: 10.145885944366455 seconds -
--- 9.856211724469997 QPS ---
```

PHẦN 5

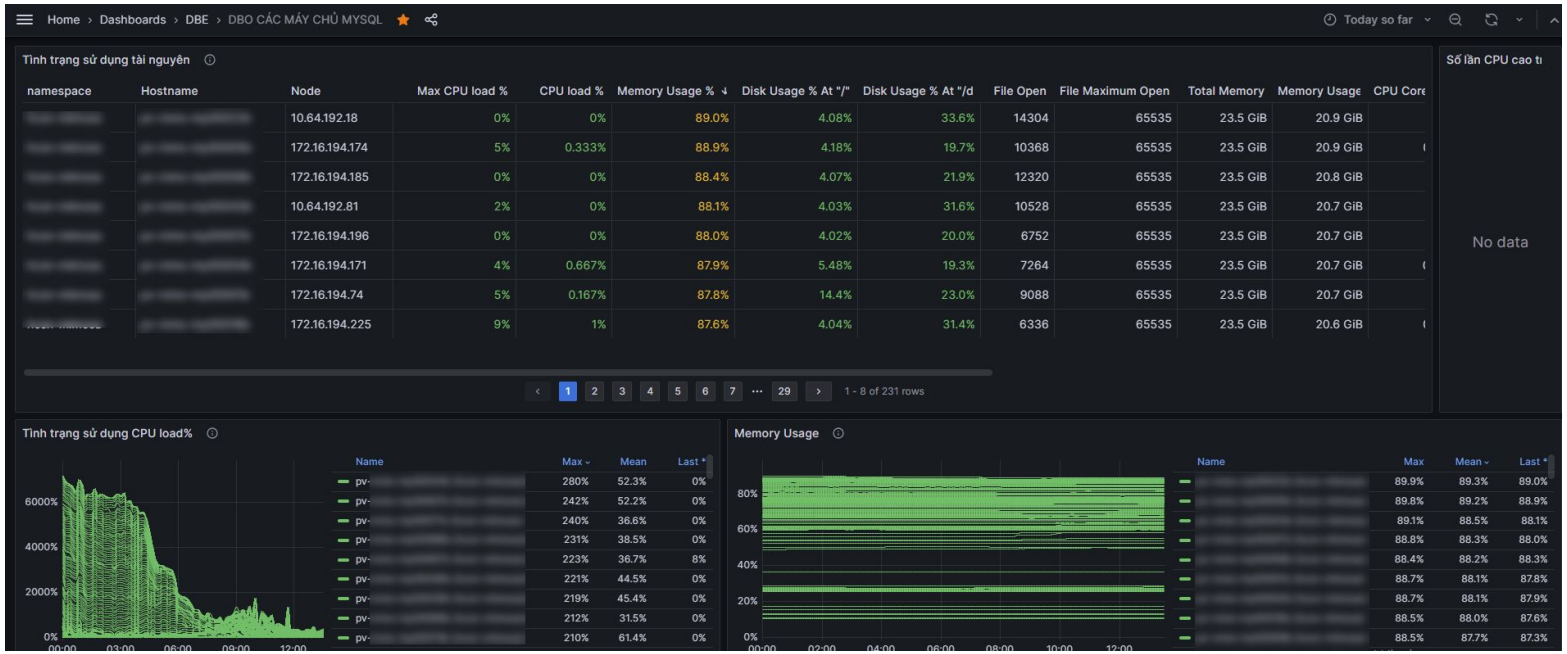
MONITOR DATABASE

Monitor database

- Tại sao cần monitor
  - Để đảm bảo duy trì ổn định hiệu năng database, một việc rất quan trọng là cần monitor thường xuyên, nhằm sớm phát hiện ra các bất thường, các nguy cơ ảnh hưởng đến hiệu năng để xử lý kịp thời
- Monitor những gì?
  - Resource máy chủ database gồm CPU, RAM, Disk I/O, Network I/O
  - Các câu truy vấn chậm hoặc tần suất gọi nhiều
  - Các câu truy vấn đang sử dụng thuật toán Table scan
  - Thừa index
  - ...

Monitor database

- Cách thực hiện
  - Tại khối sản xuất, các chỉ số monitor trên đã được hiển thị trực quan trên hệ thống monitor chung. Và việc monitor này đang được thực hiện bởi DBO các khối, dự án. Khi phát hiện vấn đề, các dev có thể truy cập để kiểm tra thêm thông tin, hỗ trợ điều tra vấn đề
  - Link: https://k8smonitor.misaonline.vpnlocal/



MISA®
TIN CẬY - TIỆN ÍCH - TẬN TÌNH

30 Năm
PHỤNG SỰ XÃ HỘI
1994 - 2024

XIN TRÂN TRỌNG CẢM ƠN!

